FIRMWARE ENGINEERING HANDOFF

EtherNet/IP Discovery and Connection Steps

ESP32 / WT32-ETH01 • UDP and TCP port 44818

IMMEDIATE GOAL	Make the ESP32 respond to a standard **ListIdentity** request, then establish a standard EtherNet/IP encapsulation session. Control and telemetry mappings are not required for this stage.

Scope: discovery and transport-session handshake only

Prepared for firmware engineering review

Stage 1 — Discover the ESP32

The Console and ESP32 must be on the same IPv4 subnet for normal broadcast discovery. The Windows firewall and network profile must allow UDP traffic on port **44818**.

Protocol used

Standard EtherNet/IP Encapsulation Protocol **ListIdentity**: command `0x0063`, sent by UDP to the subnet broadcast address on port `44818`.

1 **Open a bounded UDP discovery socket**

Console: Select the active Ethernet adapter, enable broadcast, and prepare a short response window.

Expected result: The socket can transmit and receive UDP datagrams without exposing raw network access to the renderer.

2 **Broadcast ListIdentity**

Console: Send a 24-byte EtherNet/IP encapsulation header with command 0x0063, zero session handle, and no payload to the subnet broadcast address on UDP port 44818.

Expected result: All EtherNet/IP targets on the local subnet receive the discovery request.

3 **Return the standard identity item**

ESP32 firmware: Receive the ListIdentity request and reply to the sender with a valid ListIdentity response containing an Identity Item (type 0x000C).

Expected result: The response contains the device socket address and identity fields.

4 **Validate and display the response**

Console: Check packet length, command, status, identity-item type, source address, and field bounds before accepting the device.

Expected result: A discovered-device row displays IP address, product name, revision, serial number, and compatibility status.

5 **Complete the response window**

Console: Collect unique responses for a bounded period, ignore malformed or duplicate packets, and allow a controlled retry.

Expected result: The operator can select a discovered device or enter its IPv4 address manually if broadcast discovery is blocked.

Identity values returned by the ESP32

- Encapsulation protocol version and socket address
- Vendor ID, device type, and product code
- Major/minor revision and device status
- Serial number, product name, and device state

Firmware confirmation needed

Confirm that the ESP32 listens for ListIdentity on UDP 44818 and provide either the identity values it returns or one sample response/packet capture.

Stage 2 — Establish the connection

After the operator selects a discovered device, the Console establishes an EtherNet/IP **encapsulation session**. This confirms transport-level communication; it does not yet require the application-specific CIP field mappings used for control and telemetry.

Connection used

Standard EtherNet/IP TCP connection to port `44818`, followed by **RegisterSession** command `0x0065` using encapsulation protocol version 1.

1

Select or enter the target address

Operator: Choose the ESP32 returned by discovery or enter its IPv4 address manually.

Expected result: The Console has one explicit target and does not automatically connect to an unknown responder.

2

Open the TCP transport

Console: Connect to the selected ESP32 on TCP port 44818 using bounded connect and inactivity timeouts.

Expected result: A TCP connection is established or a clear timeout/refusal error is shown.

3

Register the EtherNet/IP session

Console: Send RegisterSession (0x0065) with protocol version 1 and options 0.

Expected result: The ESP32 returns success and a non-zero session handle.

4

Validate the session response

Console: Verify command, status, sender context, protocol version, payload size, and the returned session handle.

Expected result: The UI changes to CONNECTED only after a valid response.

5

Use the registered session

Console and ESP32: Keep the TCP socket and session handle active for subsequent explicit CIP requests.

Expected result: The transport is ready; profile-specific reads and commands can be added after their CIP paths and encodings are confirmed.

6

Disconnect cleanly

Console: Send UnRegisterSession (0x0066) when possible, close the TCP socket, clear the session handle, and mark telemetry non-live.

Expected result: The app returns to DISCONNECTED without retaining a stale physical session.

Acceptance checklist for this stage

- The ESP32 appears after one ListIdentity broadcast on the same subnet.
- The displayed IP and identity match the physical unit.
- Manual IPv4 connection works when broadcast discovery is unavailable.
- RegisterSession returns a valid non-zero handle and the Console shows CONNECTED.
- Disconnect and reconnect complete without restarting either device.
- Malformed packets, timeout, cable removal, or socket closure return the Console to DISCONNECTED.

Not required yet: Enable/disable object paths, telemetry mappings, byte packing, enum values, fault codes, and watchdog details belong to the next physical test stage—not discovery or RegisterSession.